UNIVERSIDAD AUTÓNOMA DE CIUDAD JUÁREZ

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación

Campus Ciudad Universitaria



Integración y entrega continua para sistemas legado en producción: contenedorización y entornos de prueba bajo demanda sobre infraestructura autoalojada

Alumno:

Oscar Alonso Nava Rivera 214729

Requisito para la obtención del título de

INGENIERO EN SOFTWARE

Director:
Mtro. Abraham López Najera

Ciudad Juárez, Chihuahua 21 de noviembre de 2026

Introducción

[Una cuartilla: 1) campo de investigación, 2) lo ya estudiado por otros, 3) área de oportunidad, 4) objetivo del proyecto, 5) justificación.]

1. Planteamiento del problema

Una organización mantiene un sistema que sostiene gran parte de su operación administrativa y operativa, integrando sus diferentes áreas. La información se almacena en dos motores de base de datos, uno relacional y otro documental. El segundo, al no ser relacional, no implementa llaves foráneas como restricción del propio motor y, al tratarse de motores distintos, tampoco hay un mecanismo de integridad referencial que los conecte entre sí, por lo que la relación entre los datos depende de una convención y no de una restricción verificada por el sistema.

El sistema, desarrollado con Laravel 8 y PHP 8.3 y mantenido por cinco personas, nació hace dos años orientado principalmente a procesos operativos, con funciones como la generación de resguardos de unidad y bitácoras de servicio. Con el tiempo escaló hacia el área administrativa e incorporó de forma progresiva nuevas funcionalidades hasta convertirse en el sistema principal de la organización, sin que su arquitectura se revisara al mismo ritmo que su alcance. Hoy sostiene operaciones críticas, mientras que su proceso de despliegue continúa siendo completamente manual.

Cualquiera de los cinco integrantes del equipo puede realizar un despliegue, y para hacerlo es necesario acceder mediante SSH al servidor donde se aloja la aplicación, actualizar el código mediante operaciones de Git sobre el repositorio y ejecutar manualmente las migraciones pendientes, sin mecanismos de verificación automática. El historial del repositorio refleja un volumen considerable de cambios: desde su inicio se han fusionado 563 solicitudes de cambio en la rama principal, sobre aproximadamente 8,000 *commits*. Estas integraciones pueden incluir cambios que deben aplicarse sobre los motores de base de datos: modificación de esquemas en el motor relacional y, en el motor no relacional, que no tiene esquema fijo, operaciones equivalentes como la creación de índices; en ambos casos deben identificarse y ejecutarse manualmente y en el orden correspondiente.

El ritmo de integración de cambios a la rama principal no equivale a la frecuencia real de despliegue: con regularidad se agrupan varios cambios fusionados antes de llevarlos al servidor, aunque no siempre ocurre así ni existe un criterio fijo de cuántos se acumulan. En algunas semanas, incluso, se despliega casi a diario. No existe un registro sistemático del tiempo entre la fusión de un cambio en la rama principal y su despliegue real en producción; de forma informal se sabe que ese tiempo es corto, ya que no se deja código fusionado sin desplegar por periodos prolongados, pero no hay un dato preciso que lo respalde. Tampoco se conoce, con el mismo nivel de precisión, qué proporción de despliegues requiere una corrección posterior.

El proceso manual, al depender de pasos ejecutados a mano, presenta un riesgo propio de los despliegues que agrupan varios cambios: omitir alguna migración o *seeder* de los cambios acumulados. Hasta ahora no ha ocurrido, y los pocos olvidos de ejecución que se han presentado se corrigieron con rapidez y sin consecuencias relevantes.

1.1. Antecedentes

El primer elemento a considerar al intervenir un proceso de entrega de software es establecer un mecanismo sistemático y validado para medir su desempeño antes y después de la automatización. Forsgren, Humble y Kim [1] sintetizaron años de investigación empírica en miles de organizaciones y aislaron cuatro métricas clave, conocidas en la literatura como métricas DORA: frecuencia

de despliegue (*deployment frequency*), tiempo de entrega de los cambios (*lead time for changes*), tiempo de restauración del servicio (*time to restore service*) y tasa de fallos por cambio (*change failure rate*). El hallazgo central demuestra que las organizaciones de alto desempeño no sacrifican estabilidad por velocidad, sino que consiguen ambas de forma simultánea. Para este trabajo, estas métricas proporcionan un vocabulario y una definición operacional ya validados, y ofrecen un marco científico para cuantificar la mejora respecto a la línea base (*baseline*) de los despliegues manuales.

Sobre la adopción de la integración y entrega continuas (CI/CD), existe un consenso sólido respecto a las principales barreras de implementación. Revisiones sistemáticas de literatura empírica desarrolladas por Laukkanen et al. [2] y Proulx et al. [3] coinciden en que los factores humanos y organizacionales superan holgadamente a los problemas técnicos en la generación de fallas durante la liberación de código. Estos hallazgos concuerdan con lo que se observa en el sistema de estudio, cuyo principal riesgo de despliegue (omitir una migración cuando se agrupan varios cambios) depende de que una persona ejecute cada paso a mano. La literatura señala que la automatización mediante un *pipeline* establece un mecanismo restrictivo de construcción que puede reducir la posibilidad de errores humanos por omisión al hacer explícitos y verificables los pasos del despliegue.

Al intentar aplicar estos principios a sistemas *legacy*, la literatura técnica se inclina hacia una refactorización profunda o la descomposición en microservicios. Sin embargo, diferentes estudios recientes en la ingeniería de software empresarial advierten sobre los elevados riesgos y costos que puede ocasionar esta estrategia, especialmente en infraestructura restringida y autoalojada. La práctica documentada respalda la modernización incremental [4], un enfoque que prioriza la contenedorización de la aplicación tal como está, aislando sus dependencias de tiempo de ejecución (*runtime*) sin reescribir la lógica del negocio. Este es un paso crítico en contextos que operan sobre infraestructura propia y carecen de orquestación en la nube pública.

Un desafío arquitectónico particular del sistema en cuestión es su dependencia de un esquema de base de datos dual y heterogéneo (un motor relacional y uno documental simultáneos), desprovisto de un mecanismo nativo de integridad referencial. Estudios empíricos sobre entrega continua, como el de Shahin et al. [5], identifican la sincronización y la evolución de esquemas de bases de datos como uno de los cuellos de botella técnicos más severos al automatizar despliegues. Dado que no existe una gestión de transacciones distribuidas entre ambos motores, cualquier estrategia de automatización debe tratar ambas bases como una unidad lógica al ejecutar migraciones o *backfills* de datos. Esto impone una restricción de diseño importante para el *pipeline* propuesto.

Por otra parte, el obstáculo más significativo frente a la literatura tradicional de CI/CD es la validación del código. La inmensa mayoría de los trabajos académicos asumen la preexistencia de una *suite* de pruebas automatizadas como el *quality gate* (compuerta de calidad) principal. Cuando un sistema *legacy* carece de pruebas unitarias o de integración, forzar la entrega continua presenta un riesgo crítico. Para cerrar esta brecha en escenarios con datos altamente acoplados, se recomienda desplegar los cambios en entornos de prueba bajo demanda (*on-demand preview environments*), donde una persona puede validar el cambio de forma manual en condiciones parecidas a las de producción antes de integrarlo.

Operar con servidores propios (*on-premise*) añade una capa final de complejidad debido a restricciones presupuestales y a la falta de registros de contenedores (*container registries*) preexistentes. Un precedente altamente relevante en este rubro es el trabajo de Morales Agredo [6], quien diseñó en la Pontificia Universidad Javeriana Cali una propuesta de CI/CD orientada a aplicaciones *legacy*

universitarias. Su investigación demuestra que la transición hacia flujos automatizados con plataformas autoalojadas (*self-hosted*) es viable e imperativa en organizaciones con infraestructura limitada, sin depender de proveedores externos.

Al cruzar los ejes anteriores, emerge con claridad la brecha técnica y operativa que motiva este trabajo. La literatura revisada documenta sólidamente cómo evaluar el desempeño, los riesgos del error humano, la viabilidad de la contenedorización incremental y la modernización de sistemas *legacy* con plataformas autoalojadas. Ninguno de estos trabajos, sin embargo, aborda de manera conjunta la automatización de la entrega de un sistema monolítico con un esquema de datos heterogéneo, operado sobre infraestructura *on-premise* sin acceso a nube pública, y que deba garantizar la seguridad del despliegue en ausencia total de una *suite* de pruebas automatizadas. Es precisamente en la resolución simultánea de estas restricciones operativas donde se inscribe la contribución de este proyecto.

1.2. Definición del problema

Situación actual. El despliegue del sistema a producción es completamente manual y no incluye ninguna verificación automática: cada paso, incluidas las migraciones de las dos bases de datos, lo ejecuta a mano quien despliega. Los cambios se revisan entre el equipo antes de fusionarse, pero no hay pruebas automatizadas ni un entorno que reproduzca las condiciones de producción, y no se registra cuánto tardan los cambios en llegar a producción ni cuántos despliegues requieren una corrección.

Consecuencias. Cada despliegue depende de que quien lo realice recuerde y ejecute en orden todos los pasos, una tarea frecuente que se reparte entre distintas personas. Hasta ahora esto ha causado pocos problemas, pero cuando se agrupan varios cambios basta un descuido para omitir una migración. Además, la revisión entre pares es la única barrera antes de producción: lo que no detecte llega directamente a un sistema del que dependen operaciones críticas, y sin registros no hay forma de saber con qué frecuencia ocurre.

Problema central. Dificultad para desplegar y probar los cambios del sistema de forma confiable y repetible: el despliegue depende de pasos manuales que realizan distintas personas, los cambios no pueden probarse en condiciones parecidas a las de producción antes de llegar a ella y los datos están repartidos en dos bases de datos que deben actualizarse de manera coordinada.

1.3. Justificación

La automatización del ciclo de entrega de software no busca únicamente acelerar el desarrollo de la aplicación, sino reducir los riesgos de operar un sistema crítico. En el sistema de estudio, la seguridad de cada despliegue descansa en la memoria de quien lo ejecuta y en la revisión entre pares, porque no existen pruebas automatizadas ni un entorno de validación al que pueda acceder un usuario final. Mientras sea así, cualquier defecto que escape a la revisión o cualquier paso omitido termina afectando a quienes usan el sistema.

Un proceso automatizado con entornos de prueba bajo demanda atiende esta situación de tres maneras. Primero, sustituye los pasos manuales por un *pipeline* que los ejecuta siempre igual, sin

importar quién despliegue. Segundo, cuando se requiere, permite validar un cambio en una instancia de prueba antes de fusionarlo, mientras que las compuertas de calidad del *pipeline* detectan de forma automática algunos defectos desde que se sube el cambio. Ninguno de estos controles garantiza la ausencia de errores, pero aumentan la probabilidad de detectarlos antes de llegar a producción. Tercero, deja registro de cada despliegue, algo que hoy no existe y que permite evaluar el proceso con datos.

Esto beneficia al equipo técnico, al reducir su dependencia de procedimientos manuales y favorecer un proceso estandarizado, repetible y con trazabilidad. También beneficia a la organización, al disminuir el riesgo de que un error afecte la operación de un sistema que sustenta su gestión administrativa. Además, el proyecto deja documentado un caso de entrega continua sobre infraestructura autoalojada en un sistema sin pruebas automatizadas, que podría servir de referencia a equipos con condiciones similares.

1.4. Objetivo general

Diseñar e implementar un proceso de integración y entrega continua para el sistema, basado en contenedores y en entornos de prueba bajo demanda sobre infraestructura autoalojada, que reduzca los pasos manuales del despliegue y permita validar los cambios antes de su llegada a producción, y evaluarlo frente al proceso manual actual.

1.5. Objetivos específicos

Para alcanzar el objetivo general propuesto, se plantean los siguientes objetivos específicos:

- Empaquetar la aplicación en imágenes de contenedor versionadas, sin modificar su lógica de negocio, para que se ejecute de la misma forma en desarrollo, pruebas y producción.
- Implementar el despliegue de prueba bajo demanda: que cualquier integrante del equipo pueda elegir desde GitLab una rama y una instancia de prueba, y desplegar ahí la aplicación con sus dos bases de datos (MySQL y MongoDB), con opciones para aplicar migraciones y cargar datos de prueba generados mediante *seeders*.
- Configurar en GitLab, sobre infraestructura autoalojada, un *pipeline* que en cada cambio que se sube al repositorio construya la imagen y lo verifique mediante compuertas de calidad, ante la ausencia de una *suite* de pruebas automatizadas.
- Automatizar el despliegue a producción a partir de una rama de liberación (*release*), con aprobación manual, ejecutando en orden las migraciones de ambas bases de datos y verificando al final que la aplicación responde.
- Evaluar el proceso propuesto frente al proceso manual actual, comparando el número de pasos manuales, el tiempo de despliegue, el tiempo entre la fusión de un cambio y su despliegue, y la proporción de despliegues que requieren una corrección posterior.

1.6. Hipótesis

[Opcional según el tipo de trabajo.]

1.7. Alcances y limitaciones

Alcances. El proyecto se limita a un sistema, desarrollado con Laravel, MySQL y MongoDB, y cubre: la contenedorización de la aplicación sin modificar su lógica; un *pipeline* en GitLab autoalojado con compuertas de calidad; el despliegue de prueba bajo demanda en instancias de prueba; el despliegue a producción desde una rama de liberación con aprobación manual, con sus migraciones; y la comparación con el proceso manual actual.

No incluye. La reescritura del sistema ni su división en microservicios, la creación de una *suite* de pruebas automatizadas, la creación automática de instancias de prueba, la alta disponibilidad ni la actualización de versión de Laravel o PHP.

Limitaciones. Al tratarse de un solo sistema, los resultados no pueden generalizarse sin más a otros. Las compuertas de calidad no sustituyen a una *suite* de pruebas, y en la instancia de prueba los cambios se siguen validando a mano. Las instancias de prueba son limitadas en número y cada una prueba una rama a la vez. Los datos de prueba no reproducen todos los casos de uso reales. La línea base del proceso manual se reconstruye a partir de registros parciales, porque antes no se medía.

2. Marco teórico

[Teoría, conceptos y trabajo relacionado que fundamentan el proyecto. Una subsección por concepto, cada una introducida con un párrafo.]

2.1. Subtema

2.1.1. Subsubtema

3. Producto propuesto

El producto de este proyecto es un proceso de integración y entrega continua para el sistema, diseñado para operar sobre la infraestructura autoalojada de la organización. Todos sus elementos se describen en archivos versionados dentro del repositorio (infraestructura como código), por lo que el proceso es repetible y queda registrado. Se compone de:

- Las imágenes de contenedor de la aplicación y su configuración.
- Un *pipeline* en GitLab que, en cada cambio que se sube al repositorio, construye la imagen y ejecuta las compuertas de calidad.

- El despliegue de prueba bajo demanda: desde GitLab se elige una rama y una instancia de prueba, y se despliega ahí la aplicación con sus dos bases de datos, con la opción de aplicar migraciones y cargar datos de prueba.
- El despliegue a producción desde una rama de liberación, con aprobación manual, que ejecuta en orden las migraciones de ambas bases de datos y verifica que la aplicación responda.
- La documentación de uso y el informe de evaluación frente al proceso manual actual.

El proceso funciona de la siguiente manera (Figura 1). Cuando un desarrollador sube una rama, GitLab inicia un *pipeline* de revisión que construye la imagen y ejecuta las compuertas de calidad. Si el desarrollador lo requiere, puede desplegar su rama en una instancia de prueba para que el cambio se valide de forma manual, de preferencia por quien lo usará. Cuando el cambio pasa las verificaciones y las aprobaciones del equipo, se fusiona con la rama principal. Para llevarlo a producción se crea una rama de liberación a partir de la principal; sobre ella se ejecuta el *pipeline* de despliegue, que vuelve a ejecutar las verificaciones y, tras una aprobación manual, despliega la aplicación en producción.

Las características principales del proceso son:

- **Imágenes versionadas.** Las imágenes de todos los ambientes se construyen a partir de la misma definición y de la misma imagen base, lo que reduce las diferencias de configuración entre ambientes.
- **Infraestructura autoalojada.** GitLab, su registro de contenedores y los servidores donde se despliega son propios y se administran internamente.
- **Despliegue de prueba bajo demanda.** Una instancia de prueba es un servidor interno donde se despliega la aplicación para validar un cambio. Cualquier integrante puede desplegar una rama en una de ellas, con datos de prueba generados mediante *seeders* que respetan las relaciones entre MySQL y MongoDB. Las instancias son persistentes y cada una aloja una rama a la vez. La validación funcional sigue siendo manual: la instancia la hace posible y ordenada, pero no la sustituye.
- **Despliegue coordinado de las dos bases de datos.** Las migraciones de ambos motores se ejecutan en un orden definido y antes de exponer el código nuevo. Como los motores no comparten transacciones, el proceso asegura el orden y la verificación, no la atomicidad.
- **Controles en producción.** El despliegue requiere la aprobación manual de un rol autorizado y no permite borrar la base de datos.

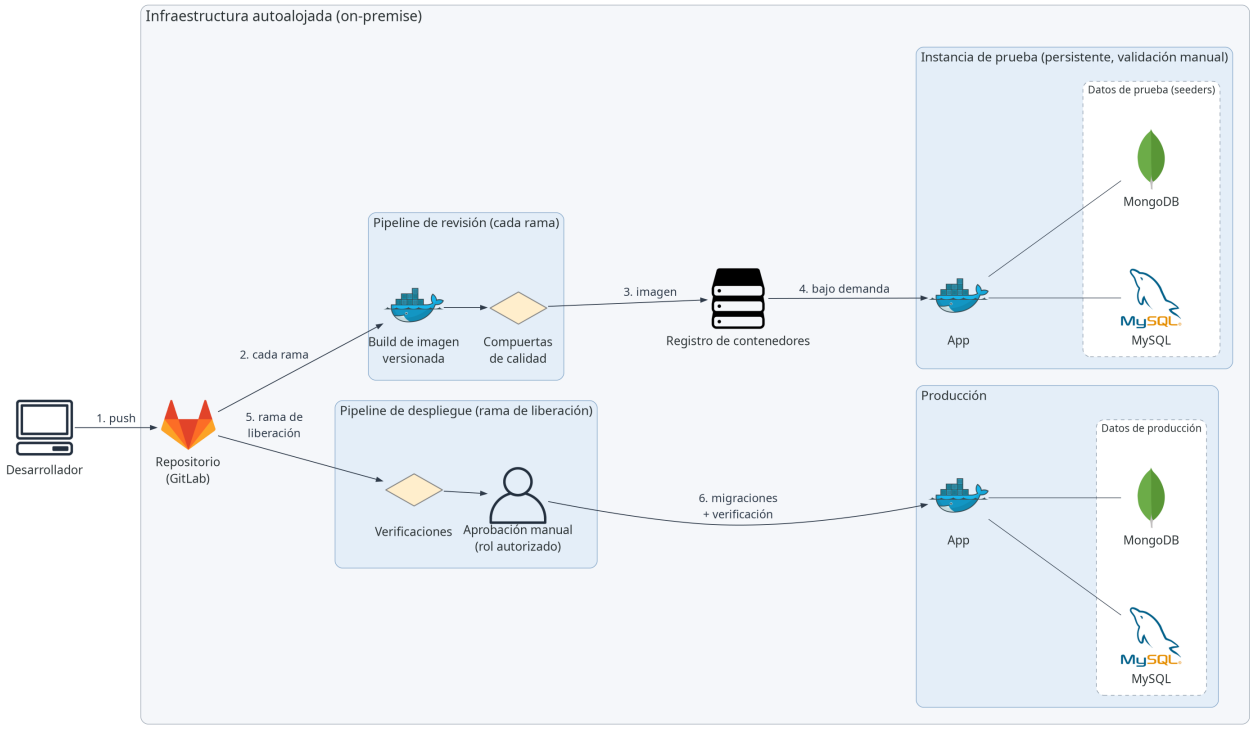


Figura 1: Arquitectura propuesta para la entrega continua.

4. Metodología

[Pasos o fases para alcanzar el objetivo, cada uno con definición y descripción de cómo se realizará.]

5. Programa de actividades

[Cronograma de actividades por periodo.]

6. Resultados esperados

[Qué se espera obtener al terminar.]

Referencias

[1] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution Press, 2018.

[2] E. Laukkanen, J. Itkonen, and C. Lassenius, "Problems, causes and solutions when adopting continuous delivery—a systematic literature review," *Information and Software Technology*, vol. 82, pp. 55–79, 2017.

[3] A. Proulx, F. Raymond, B. Roy, and F. Petrillo, "Problems and solutions of continuous deployment: A systematic review," 2018. Preprint, no arbitrado.

[4] J. Kosińska and K. Zielinski, "Cloud-native enterprise engineering: design, automation, and continuous delivery," *Software: Practice and Experience*, vol. 50, no. 8, pp. 1415–1443, 2020.

[5] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[6] A. Morales Agredo, "Propuesta de implementación de Integración Continua (CI) y Despliegue Continuo (CD) en aplicaciones *legacy* de la Pontificia Universidad Javeriana Cali," trabajo de grado, Pontificia Universidad Javeriana, Cali, Colombia, 2021.

A. Apéndice

[Material secundario o demasiado extenso para el cuerpo: código fuente completo, derivaciones, tablas de resultados completos, convenios, etc. Va después de la literatura citada y no lleva introducción ni conclusión.]